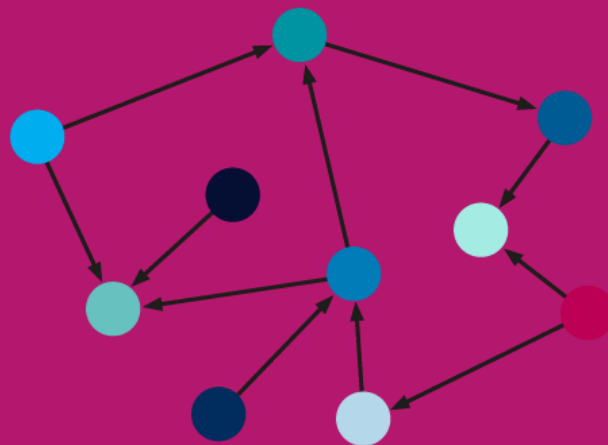
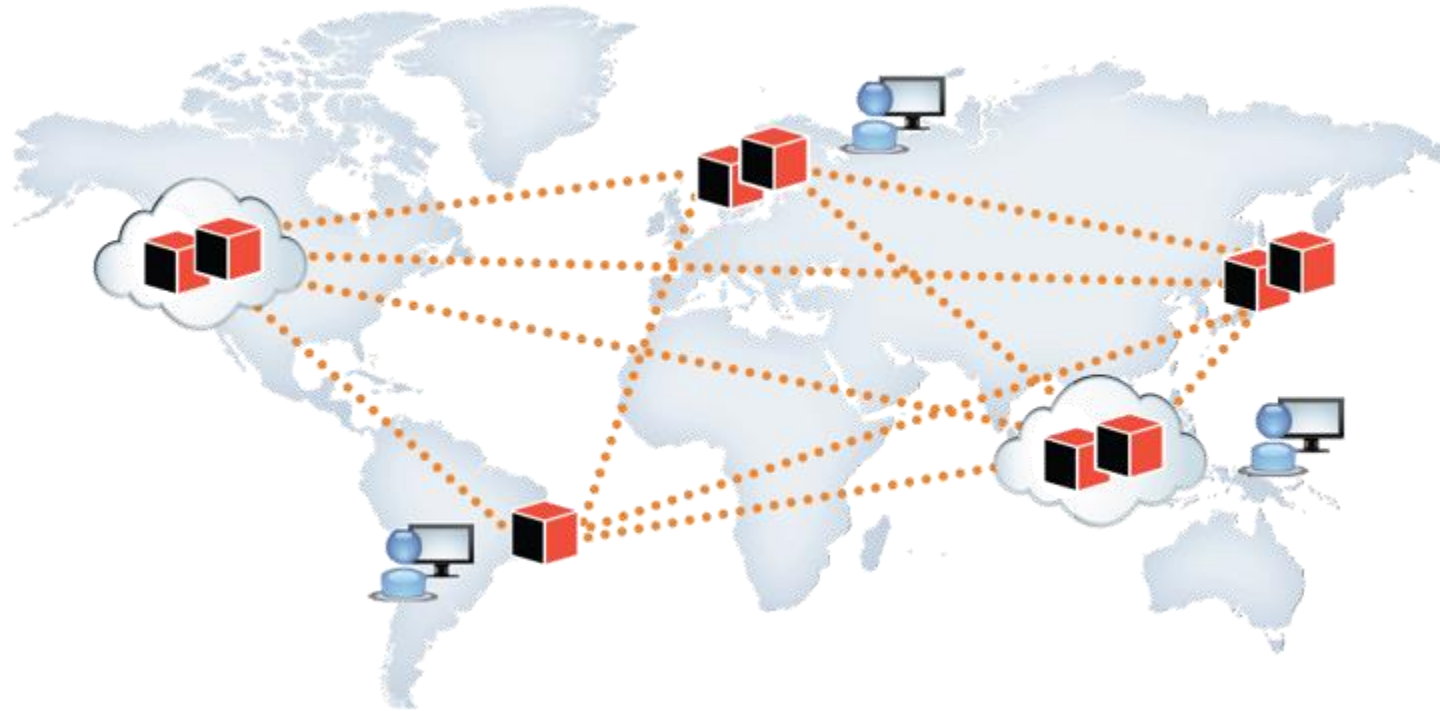




Розподілені бази даних

Жишкович Андрій Володимирович

Що таке розподілені БД?

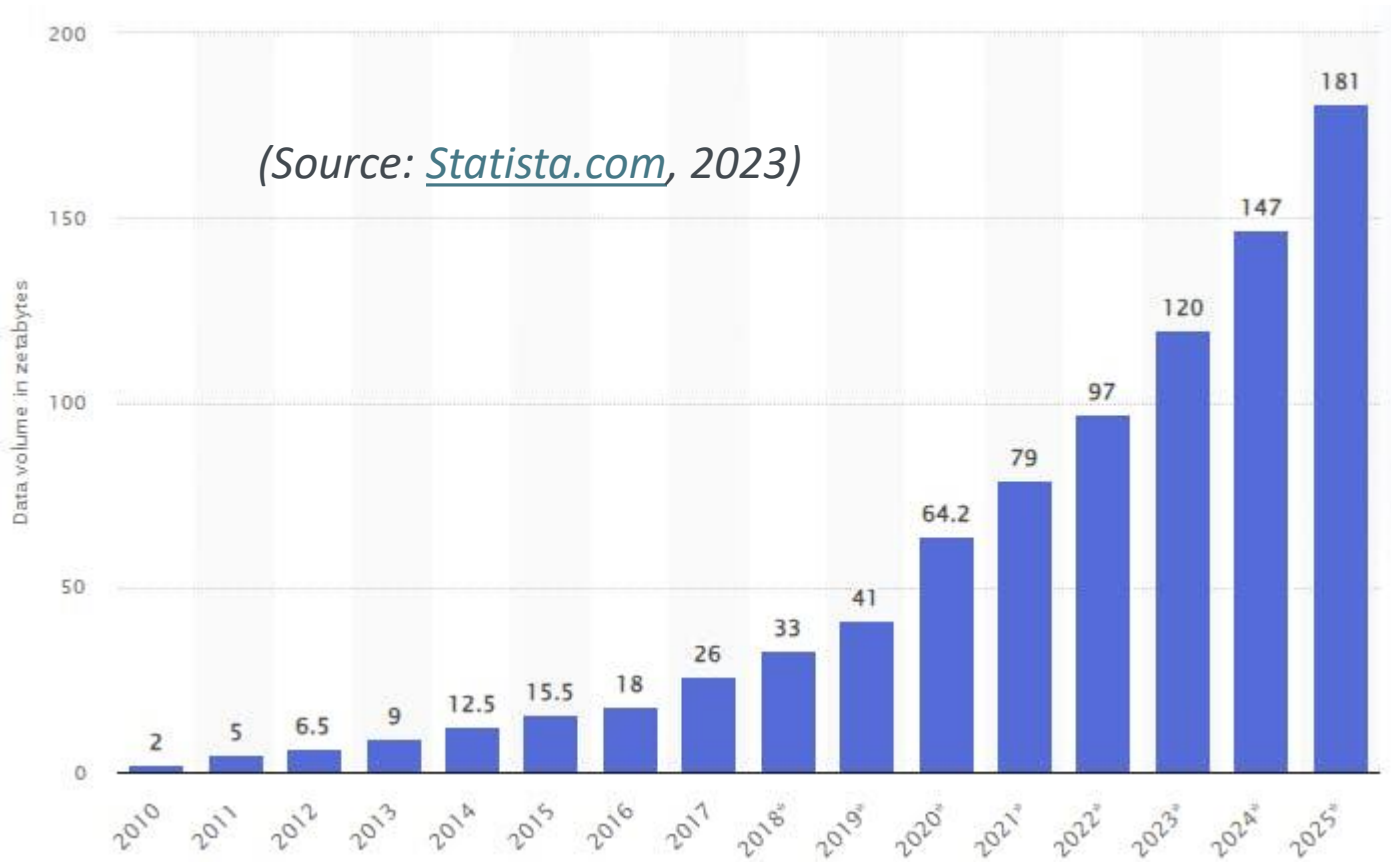


Розподілена база даних (distributed database, DDB) — це база даних, яка зберігається в кількох фізичних місцях, але виглядає для користувача як єдина логічна система.

Дані можуть бути розподілені по різних серверах, які можуть знаходитися в різних географічних місцях.

Розподілені бази даних використовуються для підвищення продуктивності, доступності та надійності системи.

Для чого?



Організації шукають гнучкі, масштабовані та високопродуктивні рішення для зберігання даних, які обробляють різні типи даних і зберігають їх цілісність.

Враховуючи доступ користувачів з мобільних пристроїв, потрібні рішення, що підтримують одночасний доступ багатьох користувачів, забезпечуючи безпеку та доступність даних.

Це робить розподілені бази даних популярним вибором для багатьох глобальних організацій.

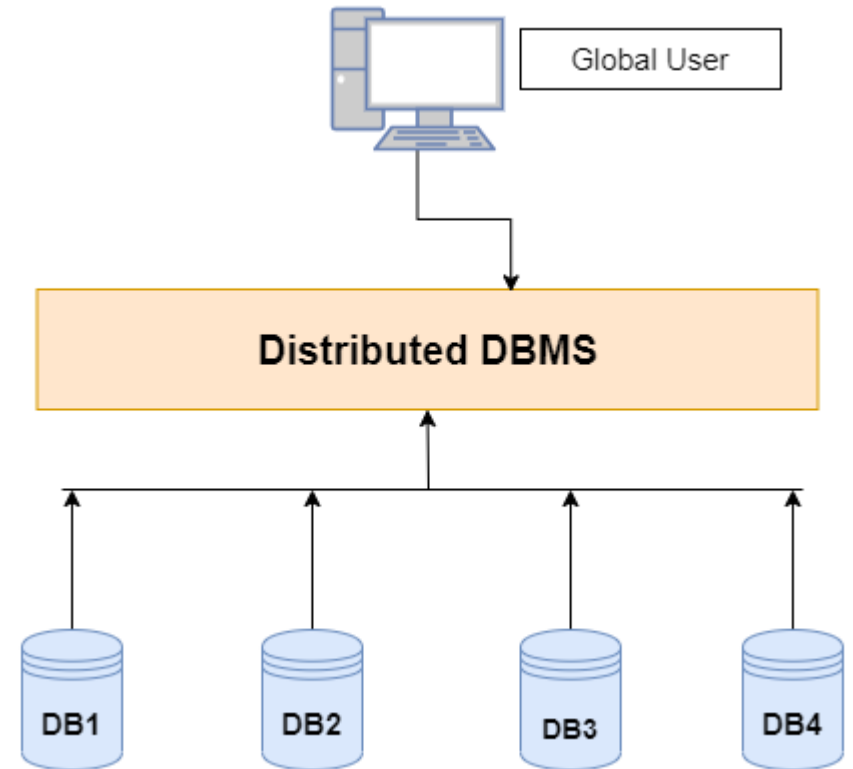
З 2020 року обсяг даних, створених, зібраних, скопійованих і спожитих у всьому світі, майже подвоївся — це **збільшення на 55,8 зетабайт лише за три роки!**

Розподілена система управління базами даних (DDBMS)

Розподілена система управління базами даних (DDBMS, Distributed Database Management System) — це система управління базами даних, яка дозволяє зберігати та обробляти дані, розподілені по кількох фізичних локаціях або серверах.

DDBMS забезпечує механізми, щоб розподіл залишався поза увагою користувачів, які сприймають базу даних як єдину БД.

Основна мета DDBMS — забезпечити узгодженість, цілісність, доступність та продуктивність даних у розподіленому середовищі.



**Як дані зберігаються
в розподіленій БД?**

Як дані зберігаються в розподіленій базі даних

Зберігання даних у розподіленій базі даних (DDBMS) включає кілька ключових концепцій і методів, які забезпечують ефективне управління даними, їх доступність та узгодженість у розподіленому середовищі.

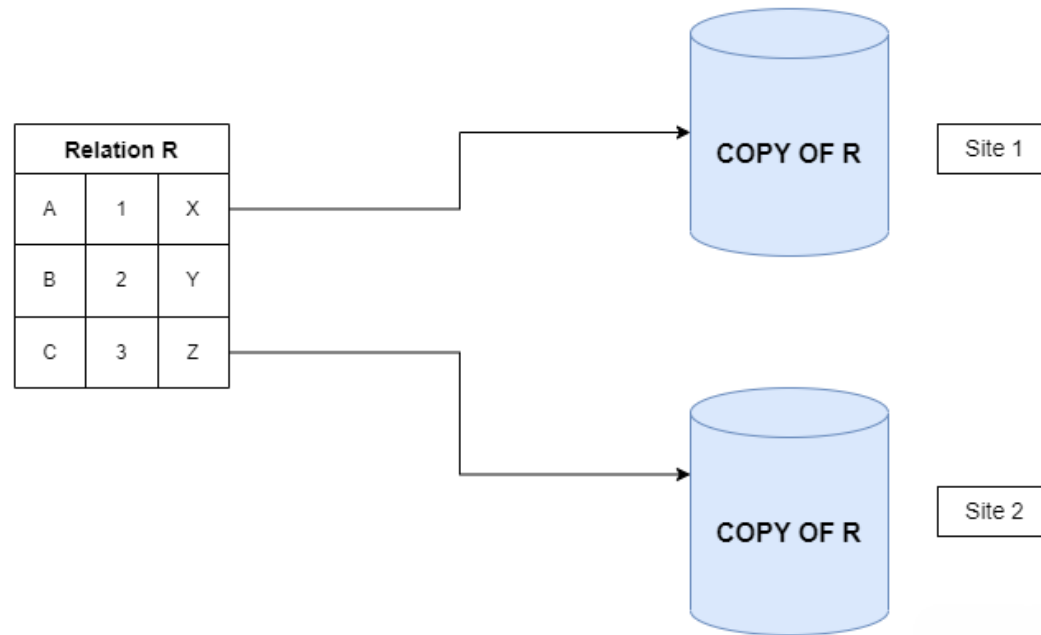
Існує 3 основні способи зберігання даних у розподіленій БД:

1. Реплікація даних
2. Фрагментація даних
3. Гібрид

Реплікація даних

Реплікація — це процес копіювання даних з одного вузла бази даних на інші вузли для забезпечення високої доступності, надійності та продуктивності системи.

У базах даних NoSQL реплікація є ключовою функцією, яка дозволяє досягти масштабованості та стійкості до відмов.



Реплікація та її види

Можливі три варіанти реплікації бази даних:

1. Повністю реплікована БД

Зберігає копії одного й того ж фрагмента БД на всіх вузлах мережі.

Всі фрагменти бази даних репліковані на кожному вузлі.

- + Доступність висока
- + Відмовостійкість висока
- Використання ресурсів неефективне
- Складність узгодженості даних між усіма вузлами

2. Частково реплікована БД

Зберігає копії тільки частини даних на кожному вузлі.

- + Ефективне використання ресурсів
- + Гнучкість в плані вибору даних для реплікації
- Складність управління задля узгодженості даних
- Відмовостійкість нижча

3. Нереплікована БД

Зберігає кожний фрагмент БД на окремому вузлі.

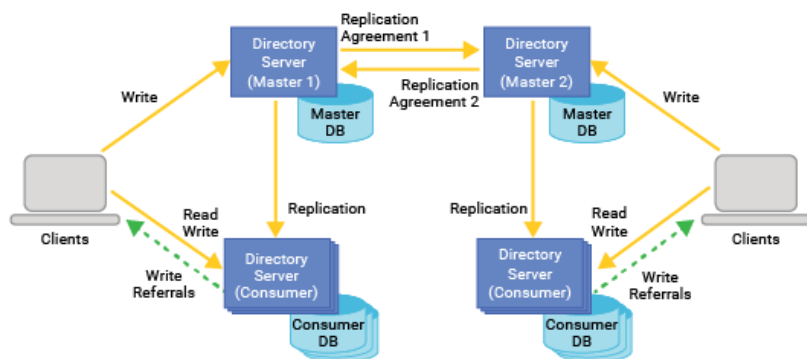
В цьому випадку дубльовані фрагменти БД відсутні.

- + Ефективне використання ресурсів
- + Простота управління
- Доступність низька
- Відсутність відмовостійкості

Реплікація та її види в базах даних NoSQL

Multi-Master Replication

based off the DynamoDB model

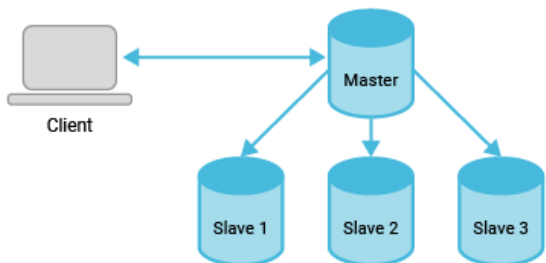


“Майстер-слейв”: майстер-вузол обробляє всі записи і реплікує дані на вузли-слейви для читання. Це масштабування читання, але вихід з ладу майстра порушує записи і вимагає нового майстра. Це одна з найпоширеніших моделей.

“Мульти-майстер”: кілька вузлів діють як майстри, дозволяючи читання та запис на будь-якому вузлі. Це забезпечує високу доступність і розподілені оновлення даних, але при виході з ладу майстра вибір нового майстра викликає короточасний простій. Ця модель є більш складною і потужною версією “Майстер-Слейв”.

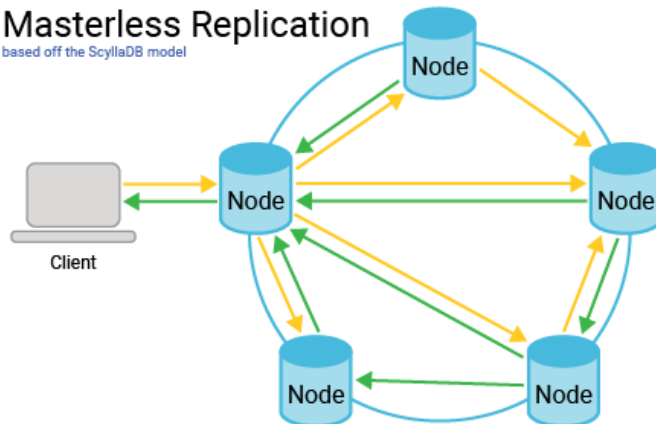
Master-Slave Replication

based off the MongoDB model



Masterless Replication

based off the ScyllaDB model

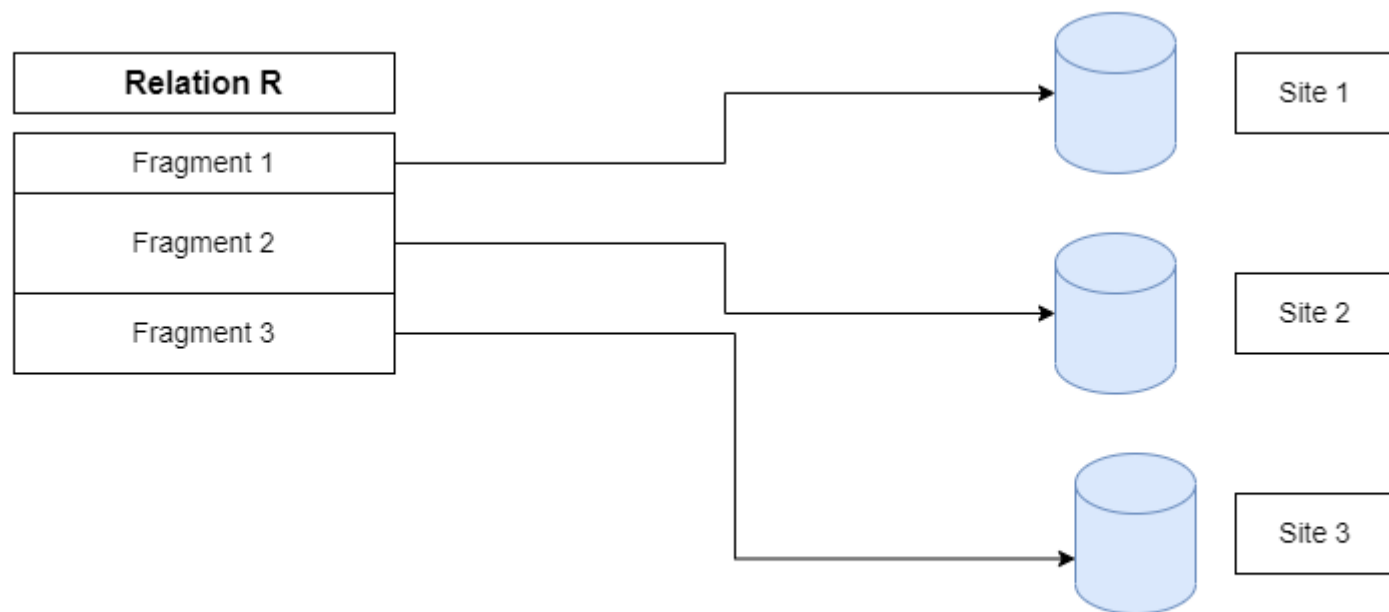


“Без майстра”: реплікують дані на кілька рівноправних вузлів, усуваючи єдині точки відмови і підвищуючи стійкість. Забезпечується ефективний розподіл даних. Зазвичай це **кворумна модель**, де для успішного запису чи читання потрібно, щоб операція була підтверджена більшістю вузлів

Фрагментація

Фрагментація — це процес розподілу бази даних на менші, більш керовані частини.

Фрагментація дозволяє підвищити продуктивність, масштабованість і доступність розподіленої бази даних.



Horizontal Fragmentation

Фрагментація

Горизонтальна: Розподіл таблиці на рядки, де кожен фрагмент містить підмножину рядків таблиці.

Вертикальна: Розподіл таблиці на стовпці, де кожен фрагмент містить підмножину стовпців таблиці.

Гібридна: Комбінація горизонтальної і вертикальної фрагментації.

This is global Table

CustomerID	CustomerName	City	Gender
1	Bob	Mumbai	Male
2	Alice	Bangalore	Female
3	Milind	Agra	Male
4	Jaya	Pune	Female

Horizontal Fragment

Fragment 1				Fragment 2			
CustomerID	CustomerName	City	Gender	CustomerID	CustomerName	City	Gender
1	Bob	Mumbai	Male	2	Alice	Bangalore	Female
3	Milind	Agra	Male	4	Jaya	Pune	Female

Горизонтальна фрагментація:

Розподіл таблиці на рядки, де кожен фрагмент містить підмножину рядків таблиці.

- + Підвищення продуктивності запитів.
- + Зменшення обсягу даних, які потрібно передавати по мережі.
- Складність у підтримці консистентності даних між фрагментами.
- Можливі проблеми з балансуванням навантаження, якщо один фрагмент стає "гарячою точкою".

Vertical Fragment

Fragment 1		Fragment 2		
CustomerID	CustomerName	CustomerID	City	Gender
1	Bob	1	Mumbai	Male
2	Alice	2	Bangalore	Female
3	Milind	3	Agra	Male
4	Jaya	4	Pune	Female

Вертикальна фрагментація:

Розподіл таблиці на стовпці, де кожен фрагмент містить підмножину стовпців таблиці.

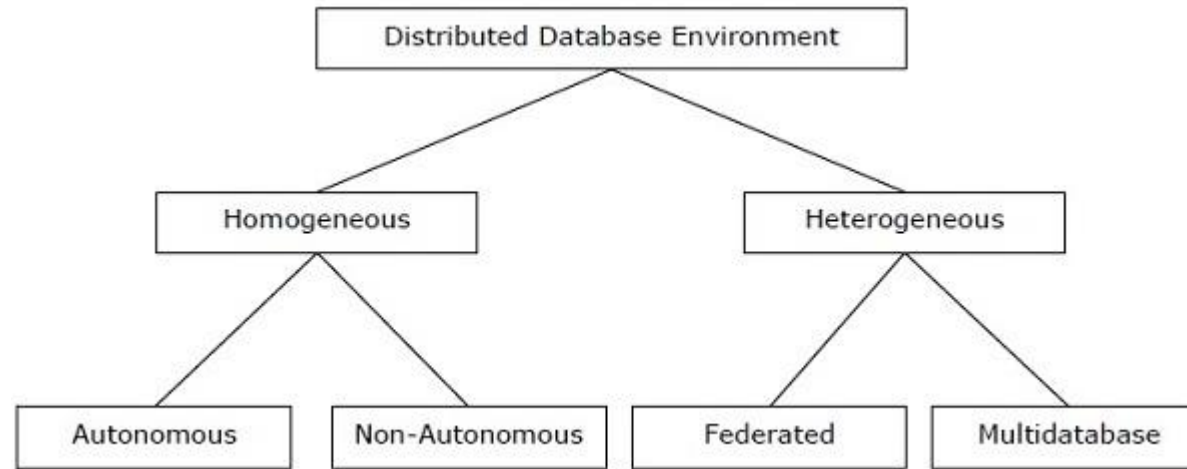
- + Підвищення продуктивності запитів, які часто звертаються до певних стовпців.
- + Зменшення обсягу даних, які потрібно передавати по мережі.
- Складність у підтримці цілісності даних між фрагментами.
- Можливі проблеми з об'єднанням даних з різних фрагментів.

Гібридне сховище

Гібридне сховище даних поєднує реплікацію та фрагментацію даних, щоб отримати переваги обох моделей

Типи розподілених СУБД

Типи розподілених СУБД



Гомогенна DDBMS

У гомогенній DDBMS всі вузли мають однакову схему бази даних і використовують однакове програмне забезпечення. Вузли знають про існування один одного і можуть безпосередньо взаємодіяти.

Такі системи простіше проектувати, керувати та розширювати, а також вони покращують паралельну обробку.

Гетерогенна DDBMS

Гетерогенна DDBMS об'єднує вже існуючі бази даних, які можуть використовувати різне програмне забезпечення і мати різні схеми. Користувачі подають запити мовою своєї локальної СУБД, а система забезпечує прозорість і сумісність між різними базами даних.

Проектування таких систем складніше через необхідність забезпечення взаємодії між різними базами даних.

Гомогенна розподілена СУБД

У гомогенній розподіленій БД **машини, вузли, сервери або сайти використовують однакову модель даних, працюють з однаковою операційною системою та використовують одну і ту ж DDBMS** або іноді кілька типів DDBMS від одного постачальника.

Приклад: Є 3 сервери університету. На всіх використовується PostgreSQL і однакова структура таблиць.

Існують два підтипи: **автономні** та **неавтономні**.

Автономні розподілені бази даних: У автономній розподіленій базі даних вузли працюють самостійно з власним повним набором даних, потребуючи лише додатка для універсальних оновлень або обміну повідомленнями між вузлами.

Неавтономні розподілені бази даних: У неавтономних розподілених базах даних вузли покладаються на централізовану систему управління базами даних (СУБД) для координації розподілу даних, комунікацій та всіх оновлень.

Зазвичай гомогенні розподілені бази даних забезпечують значний захист даних через надмірність і спрощене управління завдяки однаковості всіх вузлів.

Гетерогенна розподілена СУБД

У гетерогенній розподіленій БД **різні машини** або **сайти** можуть **зберігати різні набори даних, використовувати різні операційні системи**, містити **різні схеми даних** і потребувати програмного забезпечення для полегшення комунікації між машинами. Крім того, різні сайти можуть навіть не знати про існування інших сайтів.

Приклад: Один сервер використовує PostgreSQL, другий — MySQL, третій — Oracle.

Існують два підтипи: **федеративні** та **нефедеративні**.

Федеративні розподілені бази даних: У федеративній розподіленій базі даних кілька вузлів, які можуть функціонувати повністю самостійно і можуть містити різні дані, можуть працювати разом і функціонувати як одна сутність. Це означає, що коли відбувається запит, система визначає, який вузол найкраще підходить для відповіді, і передає запит відповідним чином. Цей процес іноді називають віртуалізацією даних.

Нефедеративні розподілені бази даних: У нефедеративній розподіленій базі даних кожен вузол працює індивідуально, і є центральний додаток, який керує доступом до кожної бази даних у кожному вузлі.

Хоча гетерогенні розподілені бази даних складніше управляти, вони пропонують більше гнучкості з точки зору моделей даних, вибору схем і типів даних, які можуть бути збережені, ніж гомогенні розподілені бази даних.

Переваги та недоліки розподілених СУБД

Переваги розподілених СУБД

Надійність: Дані можуть бути репліковані на кількох вузлах, що забезпечує доступність навіть при виході з ладу одного з вузлів.

Обмін інформацією: Користувачі одного вузла можуть отримувати доступ до даних, що зберігаються на інших вузлах.

Швидша обробка даних: Розподілена база даних дозволяє обробляти дані одночасно на кількох вузлах.

Швидший доступ до даних: Дані зберігаються там, де вони найбільше потрібні, що забезпечує швидший доступ і кращу продуктивність.

Автономність: Кожен вузол зберігає певний рівень контролю над своїми даними, на відміну від централізованої бази даних.

Модульність: Нові вузли можна додавати або видаляти за потреби, що підвищує гнучкість системи.



Недоліки розподілених СУБД

Складність: Проектування та управління розподіленими СУБД є дуже складними, особливо для гетерогенних систем, які можуть використовувати різне програмне забезпечення.

Збільшення обсягу зберігання: Реплікація даних на кількох вузлах збільшує вимоги до обсягу зберігання.

Складність підтримки цілісності: Цілісність даних означає їх узгодженість. При реплікації даних на кількох вузлах всі копії повинні бути оновлені під час зміни даних.

Витрати на комунікацію: Необхідність взаємодії між вузлами додає складності та збільшує витрати.

Безпека: Зберігання даних на кількох вузлах знижує рівень безпеки.



Типи розподілених систем

Типи розподілених систем (Master-Slave, Peer-to-Peer)

Розподілені системи — це системи, в яких обчислювальні ресурси та дані розподілені між кількома вузлами, що взаємодіють між собою для досягнення спільної мети. Існує кілька архітектур розподілених систем, серед яких найбільш поширеними є Master-Slave та Peer-to-Peer.

Master-Slave (майстер-слейв) — це архітектура розподіленої системи, в якій один вузол (майстер) контролює один або кілька інших вузлів (слейвів). Майстер відповідає за прийом і обробку запитів на запис, а слейви синхронізуються з майстром і обробляють запити на читання.

Master – Master / Multi-Master (мульти-майстер) — це архітектура розподіленої системи, в якій кілька вузлів можуть одночасно приймати записи і оновлення даних. Всі вузли є рівноправними і можуть виконувати як функції майстра, так і функції слейва.

Peer-to-Peer (P2P) — це архітектура розподіленої системи, в якій всі вузли (піри) є рівноправними і можуть виконувати як функції клієнта, так і функції сервера. Кожен вузол може ініціювати запити і відповідати на них, а також зберігати і обробляти дані.

Federated (федеративна) архітектура — це система, в якій кілька автономних баз даних об'єднані в одну логічну систему. Кожна база даних зберігає свою автономію, але може взаємодіяти з іншими базами даних у федерації.

Приклади

Приклади розподілених СУБД



Розглянемо компанію, таку як Walmart, яка має філії по всій території США. Кожна філія зберігає інформацію про клієнтів, продукти та покупки у цій філії.

Схема може виглядати наступним чином:

Customers (ID, Name, Email, Address, Phone No)

Products (ID, Name, Category, Price)

Purchases (CustomerID, ProductID, Date)



Припустимо, що генеральний директор хоче дізнатися кількість покупок по всій території США.

У ручному підході нам довелося б увійти в кожную філію, виконати запит для отримання кількості покупок і потім об'єднати результати.

Це може зайняти дуже багато часу.

Але якщо система є розподіленою базою даних, ми можемо отримати кількість всіх покупок за допомогою одного запиту.

Комерційне програмне забезпечення для розподілених СУБД

Існує багато комерційних програмних продуктів для розподілених СУБД для NoSQL



Ці системи забезпечують ефективне управління даними в розподілених середовищах, підвищуючи продуктивність, масштабованість та надійність додатків.

Amazon SimpleDB

ДЯКУЮ ЗА УВАГУ!